

Cómo se ve un buen prompting de Claude de cara al examen

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 8 de 12 · Notas en español (es-ES)

La segunda mitad del 19 de agosto fue una lección de prompting: sed **claros y específicos**, preferid **instrucciones a preguntas divagantes**, usad **verbos de acción**, añadid **pautas** de longitud y restricciones, marcad los datos con **etiquetas estilo XML** y luego **evaluad** cada plantilla. Vijay planteó la evaluación como trabajo de quien desarrolla IA, con un bucle de cinco pasos al estilo Claude y tres evaluadores: humano, código o LLM-as-judge.

Lo claro gana a lo largo

Más palabras no son más precisión. Una divagación vaga sobre paneles solares produce respuestas vagas. Reescribid: «Escribe tres párrafos sobre cómo funcionan los paneles solares». Empezad por la tarea. Verbos que le gustaban: write, generate, produce, identify, create, plan. Preferid «Identifica tres países que usen energía geotérmica; incluye estadísticas de generación de cada uno» a un relato de que algo «suena bien».

Un prompt de relato corto sin restricciones también es flojo. La misma tarea más pautas — menos de 1000 palabras, una acción que revele el talento, al menos un personaje secundario — es la versión lista para examen. Las restricciones no son adorno; son la especificación.

Estructura con sabor Claude

- Etiquetas XML (inventadas por vosotros) para documentos, normas y entrada no fiable.
- Prefill + secuencias de parada cuando el consumidor necesita un sobre cerrado (JSON, vallas).
- Ejemplos few-shot dentro del prompt si el formato es frágil.
- Parámetro system para el rol permanente / anti-alucinación (módulo 07), contenido user para la instancia.
- Temperature baja al extraer; más alta al redactar prosa.

El bucle de evaluación (cinco pasos en clase)

1. **Borrador** de prompt (incluso un ingenuo «Please answer the user's question: {q}»).
2. **Conjunto de eval** de preguntas realistas — incluidas tareas AWS del relato de laboratorio, más basura fuera de distribución («qué es 2+2», gachas) si necesitáis ver rechazos o alcance.
3. **Ejecutad** el conjunto; guardad pares prompt→respuesta.
4. **Puntad** — no solo un booleano: una nota (p. ej. sobre 10) de completitud, formato, fidelidad.
5. **Revisad** el prompt (y quizá temperature / modelo) y repetid. Las notas pueden *bajar*; eso también informa.

El escenario de herramienta interna AWS: ingeniería describe un recurso en inglés; Claude devuelve Python, JSON o regex. Las preguntas de eval deben incluir peticiones correctas, erróneas y regulares para ver cómo se comporta el borrador antes de añadir «answer with ample detail» y otras elaboraciones.

Tres formas de puntuar

Evaluator	Sirve para	Notas de coste / sesgo
Humano / SME	Calidad de traducción, tono de marca	Lento; necesario cuando no podéis fijar una métrica
Código (Pydantic, regex, JSON schema, tests)	Forma, tipos, «exactamente una frase»	Barato, repetible; no sabe si los hechos son ciertos
LLM-as-judge	Rúbricas (fidelidad, calidad de razonamiento) a escala	Usad un modelo capaz; pedidle que <i>no</i> sustituya su conocimiento del mundo; sigue habiendo sesgo; no hace falta un juez anidado

El modelo juez debe actuar como una «constitución»: aplicar vuestra rúbrica, no reescribir la respuesta con su propio francés. Afináis el *prompt del juez*; no entrenáis un modelo nuevo para Foundation. Preferid jueces de clase Opus frente a un generador minúsculo si hay que juzgar lengua — pero el aviso de coste de Vijay: si todo es un LLM, la factura explota. Implementad primero evaluadores de código. La plataforma de Anthropic tiene utillaje de eval para desarrolladores; las evaluaciones de AWS Bedrock se mostraron como análogo (Nova como juez, modelo pequeño como generador, eval humana como servicio de pago). Claude no os quita la necesidad de vuestro propio bucle.

Messages frente a system — redacción de examen

Vijay dijo que los exámenes preguntan «sobre todo» cómo se configuran **messages** y el **system prompt** en `create`. User y assistant viven en el array; system es un parámetro hermano. No inventéis un `role`: `system` dentro de `messages` salvo que la documentación vigente acepte esa forma — el código de clase usó el campo de primer nivel.

La eval de agentes es más grande

El mismo bucle borrador→dataset→nota aparece en tests de agentes, pero los agentes añaden tools, RAG y orquestación. En respuestas Foundation dejad la eval de prompts en el ámbito de desarrollo salvo que la pregunta nombre tools.

LAS UI DE EVAL DE PLATAFORMA CAMBIAN

Las URL de «developer tests» y los clics de Bedrock del 19 de agosto caducan. Los conceptos quedan: dataset, modelo generador, rúbrica de juez. Priman las **docs de eval / guardrails** y de prompting.

Rúbrica de bolsillo para respuestas Foundation

- Tarea primero, restricciones después, datos en etiquetas.
- Mostrad el formato que queréis (o haced prefill).

- Decid cómo mediríais el éxito con un evaluador.
- Nombrad stop_reason / max_tokens si la salida puede truncarse.
- Citad la documentación vigente de prompting de Anthropic, no una anécdota de transcripción, si la técnica ya tiene nombre de producto (structured outputs, extended thinking).

Documentación oficial

- **Visión general de prompting**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Claridad, etiquetas XML, ejemplos — clave de corrección de este capítulo.

- **Salidas estructuradas**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Cuando el examen menciona JSON garantizado.

- **Guardrails / eval**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Mentalidad de evaluación más allá del camino feliz.

- **Messages API**

<https://docs.claude.com/en/api/messages>

Configuración de system frente a messages.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.